AMD CPU PoC — MariaDB Core-Scaling Benchmark

README & Procedure — amd-mariadb-poc-v1.3.sh

2026-07-24

Contents

Overview

`amd-mariadb-poc-v1.3.sh` is an interactive, customer-facing Proof-of-Concept runner that demonstrates AMD EPYC CPU core scaling for MariaDB using the HammerDB TPC-C OLTP workload (and optionally TPC-H analytics). It is a wrapper around the base benchmark runner in skariapaul/mariadb-tpcc-bench (auto-cloned on first run), so this single script is the only thing you need to copy onto a benchmark host.

v1.3 adds **scale-out mode** on top of everything v1.2 did: every eligible core count is now benchmarked both as a single MariaDB instance and as N independent containers running concurrently, so a PoC can show what horizontal scale-out recovers from single-instance lock/latch contention at high core counts. `amd-mariadb-poc-v1.2.sh` is kept in the repo unchanged for anyone already relying on its single-instance-only behavior; this document covers v1.3.

What the wrapper adds on top of the base runner

- **Interactive PoC wizard** — prompts for scale-out shard size, core counts, NUMA pinning, run sizing, and report options, with sane AMD-aware defaults for every field.
- **Scale-out mode** (new in v1.3, on by default):

  - Every core count that's an exact multiple of `--shard-cores` (default 8) and greater than it is benchmarked **twice** — once as a single MariaDB instance, once as N = cores / shard-cores independent containers running concurrently, throughput summed.
  - Shards get disjoint, NUMA-aware CPU pins, their own warehouse count (auto-sized to 10x the shard size unless `--warehouses` is given explicitly), and an even split of the configured buffer pool.
  - The default core-count list is every multiple of `--shard-cores` up to the host's actual core count — not capped — so bigger SUTs sweep further automatically (e.g. `8 16 24 32 40 48 56 64` on a 64-core host).
  - Disable with `--no-scale-out` to reproduce v1.2's single-instance-only sweep exactly.
- **AMD-aware host tuning** (optional, needs sudo, auto-reverted on exit):
  - CPU frequency governor → `performance`
  - `vm.swappiness` → `1`
  - Transparent hugepages → `never`, with explicit 2 MiB hugepages sized to the InnoDB buffer pool (the base runner then enables MariaDB `large_pages=ON`)
- **Full system inventory capture** — CPU/NUMA/memory/kernel/BIOS details saved alongside the results for the report.
- **Per-run measurement extras** (captured separately for single-instance and scale-out phases):
  - CPU utilisation sampling via `mpstat`
  - CPU package power via RAPL (`/sys/class/powercap`), wrap-safe sampled every 2 seconds — used to compute NOPM/Watt "performance-per-watt"
- **Scaling-efficiency analysis** — speedup vs. the smallest single-instance core count tested, and % scaling efficiency relative to ideal linear scaling, computed per row (single-instance or scale-out).
- **Consolidated report** in Markdown, with an optional PDF rendered via pandoc + LaTeX, containing:
  - Executive summary with headline single-instance speedup, and (if any scale-out ran) the aggregate uplift at the largest core count
  - System-under-test inventory table
  - TPC-C scaling table with a Mode/Containers column — one row per (core count, mode): NOPM, speedup, efficiency %, watts, NOPM/W
  - ASCII throughput chart (single and scale-out bars labeled separately)
  - Transaction latency percentiles and TPC-H results (single-instance runs)
  - Appendix A: every generated single-instance `my.cnf`, one per core count
  - Appendix A2: the scale-out shard `my.cnf` (fixed size, one shown)
  - Appendix B: raw `lscpu` / memory details

Prerequisites

Run on the benchmark host itself (bare metal or a VM with the cores you want to test).

```
# Ubuntu/Debian
sudo apt-get install -y docker-ce docker-compose-plugin git libmariadb3 sysstat numactl
# for PDF report output:
sudo apt-get install -y pandoc texlive-latex-recommended texlive-fonts-recommended

# RHEL/Rocky
sudo yum install -y docker-ce docker-compose-plugin git mariadb-connector-c sysstat numactl pandoc
```

Also required:

- Docker daemon running and reachable (`docker info` succeeds)
- User is in the `docker` group, or can `sudo`
- ~20 GB free disk for container images, datadirs, and results (more if scale-out runs many shards concurrently — each shard is a full datadir)
- (Optional but recommended) sudo/root access, for host tuning and readable RAPL power counters

Scale-out mode additionally needs a `mariadb-tpcc-bench.sh` checkout that supports `--container-name` and `--tmp-dir` (so concurrent shard containers don't collide on Docker container name or HammerDB's `/tmp` job files). Point `--base-dir` at such a checkout. Without it, the script will detect the missing support and stop with

a clear message — either supply a patched `--base-dir`, or pass `--no-scale-out` to run against a vanilla clone.

The script itself checks all of this in a pre-flight checklist and will offer to auto-install pandoc + LaTeX if missing and sudo is available.

Command-line Options

All options are optional — the interactive wizard will ask for anything not supplied on the command line.

Option	Description	Default
`--cores "8 16 24 32"`	Core counts to test (space-separated)	multiples of `--shard-cores` up to the host's core count
`--shard-cores N`	Cores per scale-out container; also sets the step of the default core list	8
`--no-scale-out`	Single-instance only (skip scale-out rows)	scale-out enabled
`--warehouses N`	TPC-C warehouses for single-instance runs (recommend >= 10x max cores)	wizard recommends max_cores x 10
`--rampup N`	Ramp-up minutes per run	2
`--duration N`	Timed-run minutes per run	10
`--tpch`	Also run TPC-H analytics	off
`--tpch-sf N`	TPC-H scale factor	1
`--buffer-pool SIZE`	InnoDB buffer pool, e.g. `64G` (total; scale-out splits evenly across shards)	75% of host RAM
`--mariadb-ver VER`	MariaDB Docker image tag	11.4
`--numa-node N`	Pin runs to NUMA node N (auto cpuset per core count/shard)	none (or wizard prompt if host has >1 NUMA node)
`--no-host-tuning`	Skip governor/hugepage/swappiness tuning	tuning enabled
`--report md|pdf|both`	Report format	both
`--customer "Name"`	Customer name printed on the report cover	"AMD EPYC PoC"
`--title "Text"`	Report title override	"MariaDB Core-Scaling Benchmark — AMD CPU Proof of Concept"
`--workdir PATH`	Output directory	`./amd-poc-YYYYMMDD-HHMM`
`--base-dir PATH`	Path to an existing `mariadb-tpcc-bench` checkout	auto-clone next to workdir
`--dry-run`	Show the plan, run nothing	off
`--yes` / `-y`	Non-interactive, accept defaults/flags	off
`-h` / `--help`	Show usage	—

Note: `--warehouses`, when passed explicitly, applies uniformly to every scale-out shard as well as the single-instance runs. Left unset, shards auto-size to `10 x --shard-cores` independently of the single-instance warehouse recommendation.

Procedure — Running the Benchmark

Step 1: Copy the script to the benchmark host

```
scp amd-mariadb-poc-v1.3.sh user@benchmark-host:~/
ssh user@benchmark-host
```

```
chmod +x amd-mariadb-poc-v1.3.sh
```

For scale-out mode, also make a patched `mariadb-tpcc-bench` checkout available on the host (or clone it and pass `--base-dir` to it) — see Prerequisites above.

Step 2: (Optional) Sanity-check the plan with a dry run

This walks through the wizard (or applies `--yes` defaults) and prints the run plan and time estimate — including which core counts get a scale-out pass — without executing anything or touching host settings.

```
bash amd-mariadb-poc-v1.3.sh --dry-run --yes --base-dir /path/to/mariadb-tpcc-bench
```

Step 3: Run the interactive wizard (recommended for a customer PoC)

```
bash amd-mariadb-poc-v1.3.sh --base-dir /path/to/mariadb-tpcc-bench
```

The wizard will:

1. Print detected CPU model, socket/core/NUMA topology, RAM.
2. Ask for the scale-out shard size (default 8 cores) — this also sets the step of the suggested core-count list.
3. Ask for core counts to benchmark (e.g. `8 16 24 32`, defaulting to multiples of the shard size up to the host's core count).
4. If the host has multiple NUMA nodes, ask whether to pin the run to one node (recommended when the largest core count fits in a single node).
5. Confirm whether to also run scale-out benchmarks (on by default) — explains that this roughly doubles run time for eligible core counts.
6. Recommend and confirm a TPC-C warehouse count for single-instance runs (>= 10x the largest core count tested, to avoid hot-row contention masking true CPU scaling); notes that scale-out shards auto-size independently unless `--warehouses` is given explicitly.
7. Ask for ramp-up/timed-run minutes, whether to also run TPC-H, InnoDB buffer pool size (total, split across shards in scale-out mode), and MariaDB version.
8. Ask whether to apply host tuning (governor, swappiness, hugepages) — needs sudo; all changes are reverted automatically when the script exits.
9. Ask for the customer name and report format, print the full run plan and a rough time estimate (accounting for the extra scale-out passes), then ask for final confirmation before starting.

Step 4: Or run fully non-interactively

Useful for scripted/repeatable PoCs, e.g. a 64-core EPYC PoC sweeping 8..64 in steps of 8, single-instance **and** scale-out (8c shards) for every eligible core count:

```
bash amd-mariadb-poc-v1.3.sh --yes \
  --base-dir /path/to/mariadb-tpcc-bench \
  --warehouses 640 --rampup 3 --duration 15 --buffer-pool 96G \
  --numa-node 0 --mariadb-ver 11.4 \
  --customer "Acme Corp" --report both
```

Single-instance only (v1.2-equivalent behavior), or a custom shard size:

```
bash amd-mariadb-poc-v1.3.sh --yes --no-scale-out --cores "8 16 32 64"

bash amd-mariadb-poc-v1.3.sh --yes --shard-cores 16 \
  --base-dir /path/to/mariadb-tpcc-bench
```

Step 5: Let it run

For each core count in the sweep, the script runs a **single-instance** phase, and then — if that core count is an exact multiple of `--shard-cores` and greater than it — a **scale-out** phase:

Single-instance phase:

1. Starts background `mpstat` and RAPL power sampling.
2. Invokes the base runner (`mariadb-tpcc-bench.sh`) to build a fresh schema at that core budget and run the timed TPC-C (and optional TPC-H) test.
3. Stops sampling and computes average package power for that config.

Scale-out phase (when eligible):

1. Computes N disjoint, NUMA-aware CPU pins of `--shard-cores` each.
2. Starts background `mpstat` and RAPL power sampling for the whole batch.
3. Launches N base-runner invocations concurrently — each its own container, port, workdir, and HammerDB temp directory — building its own schema and running its own timed TPC-C test in parallel.
4. Waits for all shards, sums NOPM/TPM/VUs across shards that completed successfully, and writes the aggregate.

Expect roughly `(schema-build-time + rampup + duration + 5 min overhead)` minutes per phase — the script prints its own combined estimate before starting. Schema build is ~15 seconds per warehouse and is repeated for every phase (shards build concurrently with each other, but sequentially after that core count's single-instance phase).

Step 6: Collect the report

On completion the script prints the output layout and consolidates everything into a report:

```
amd-poc-YYYYMMDD-HHMM/
  amd-mariadb-poc-report.md    - consolidated PoC report (source of truth)
  amd-mariadb-poc-report.pdf   - rendered via pandoc, if available
  bench/                       - single-instance base runner workdir
    configs/  *.cnf per core count
    results/  raw HammerDB logs + per-run summaries
  bench-scaleout/<N>core/shard<i>/  - one isolated workdir per scale-out shard
  metrics/<N>core/single/      - power.txt (RAPL), mpstat.log
  metrics/<N>core/scaleout/    - power.txt, mpstat.log, aggregate_summary.txt,
                                 shard<i>.log, tmp-s<i>/
  sysinfo/                     - lscpu, numactl, free, os-release, ...
```

Hand the customer `amd-mariadb-poc-report.pdf` (or `.md`); keep the rest for your own backup/troubleshooting.

PoC Tips

- **Warehouses >= 10x the largest core count** for single-instance runs. The wizard defaults to this. Too few warehouses causes hot-row contention that flattens the curve and undersells the CPU.
- **Scale-out rows are N independent MariaDB instances**, each with its own schema, buffer-pool slice, and CPU pin, benchmarked concurrently and summed — a proxy for a sharded/horizontally-scaled deployment, not one logically-consistent database. Say so explicitly if a customer asks.
- Scale-out shards use ports starting at 3400 (vs 3307 for single-instance runs) and container names like `mariadb-bench-32c-s0`, so they never collide even though they run at the same time.
- Single-instance always runs before scale-out for a given core count, and the smallest core count in the sweep is always single-instance only — this guarantees HammerDB is downloaded/cached before any concurrent scale-out shards launch, avoiding a race on the shared `~/HammerDB` install on a brand-new host.
- **Pin to one NUMA node** with `--numa-node` when the largest core count fits in a single node — this removes cross-socket memory latency from the comparison, for both single-instance and scale-out phases. Check topology first with `numactl --hardware`.
- On EPYC, NUMA node cpulists enumerate physical cores before SMT siblings, so NUMA-derived cpusets naturally land on physical cores first.
- Use `--duration 15` or more for headline numbers; short (5-minute) runs are fine for dry-fitting the setup, but understate steady-state throughput.
- RAPL power needs readable `/sys/class/powercap/*rapl*` counters — run as root (or via sudo) if NOPM/Watt shows as `-` in the report.

- RAPL power only sums top-level package zones — Intel exposes per-package core/uncore sub-zones under `/sys/class/powercap` that are subsets of the package total, not separate draw; summing them too would double/triple the reading (this previously inflated readings 2-4x on Xeon hosts). With `--numa-node` set, power is further scoped to that NUMA node's physical package(s) only, so a pinned run doesn't also report an idle socket's draw — the scaling-table footnote in the report states whether the node-to-package mapping was resolved or it fell back to whole-host power.
- Durability is intentionally relaxed for peak-throughput measurement (`innodb_flush_log_at_trx_commit=0`, `innodb_doublewrite=0`); the report states this explicitly so the customer sees the methodology honestly.
- All host tuning is reverted automatically on exit, including on Ctrl-C.

Troubleshooting

Symptom	Likely cause / fix
Docker not found	Install Docker; the script exits immediately.
Docker daemon not reachable	`sudo systemctl start docker`, or add your user to the `docker` group.
`libmariadb.so.3` missing	`sudo apt-get install -y libmariadb3` (Ubuntu/Debian) or `sudo yum install -y mariadb-connector-c` (RHEL/Rocky).
"Scale-out mode needs a base runner with –container-name/–tmp-dir support"	Point `--base-dir` at a patched `mariadb-tpcc-bench` checkout, or pass `--no-scale-out`.
"N/M scale-out shards … exited non-zero"	Check `metrics/<N>core/scaleout/shard<i>.log` for that shard's error (often insufficient RAM/CPU for the shard count requested, or a port already in use from a leftover container).
PDF not generated	`pandoc` or a LaTeX engine missing — install `pandoc texlive-latex-recommended texlive-fonts-recommended`, or re-run with `--report md`. Check `pandoc.err` in the workdir.
NOPM/Watt column shows `-`	RAPL energy counters not readable — re-run as root/sudo.
"Could not resolve NUMA node N to a physical package" note in the report	`/sys/devices/system/cpu/cpu*/topology/physical_package_id` wasn't readable (uncommon; some virtualized/restricted environments) — power then falls back to summing every package on the host instead of just the pinned node's.
Container "did not start within 3 minutes"	Usually an invalid `--buffer-pool` value missing a unit suffix (e.g. `16` instead of `16G`) — MariaDB then requests far too little memory and fails to start. Also check that the per-shard buffer-pool split (total / shard count) still leaves each shard enough memory.
Core count rejected	Requested core count exceeds host logical CPU count (`nproc`).
Scale-out skipped for a core count you expected	That core count isn't an exact multiple of `--shard-cores` — the script warns and runs single-instance only for it. Adjust `--cores` or `--shard-cores`.

Reference

- Base benchmark runner: https://github.com/skariapaul/mariadb-tpcc-bench
- Workload: HammerDB TPC-C (OLTP) and optional TPC-H (analytics)

- Script version documented here: **v1.3** (adds scale-out mode). `amd-mariadb-poc-v1.2.sh` and its walk-through (`amd-mariadb-poc-v1.2-README.pdf`) remain available unchanged for single-instance-only use.